

■ Architectural Decision Log — GCP Marker Project

1. Which Model Selection Strategy

- Step 1 - Normal Image Classification (CNN):** *Concept:* Crop regions and pass them to standard classifiers. *Rejection Reason:* Completely incapable of predicting spatial coordinates. Because targets appear randomly, a standalone classifier cannot extract or output the required (x,y) pixel values.
- Step 2 - Segmentation Models:** *Concept:* Predict pixel-level masks for each marker shape. *Rejection Reason:* Excessive annotation and computational overhead. Our dataset only provides center points, not pixel masks, making mask-level architectures over-engineered for a point localization task.
- Step 3 - RAG / Multimodal VLM Networks:** *Concept:* Pass the drone image to large vision models to extract coordinate text tokens via prompting. *Rejection Reason:* Highly inefficient and slow. VLMs cannot run natively on edge drone systems and struggle with sub-pixel absolute coordinate regression on 4K imagery.
- Step 4 - Object Detection Frameworks (YOLO Baseline):** *Concept:* Detect bounding regions and classify shapes simultaneously. *Decision:* Selected as our initial baseline model (YOLOv8s). It runs end-to-end, handles small objects efficiently, and provides a quick operational benchmark.
- Step 5 - Keypoint Detection Models (YOLO Pose):** *Concept:* Deploy a specialized regression head to target individual (x,y) coordinates directly via point loss minimization. *Decision:* Selected as our primary production model (YOLOv8n-pose). If standard YOLO bounding box center calculations do not yield enough coordinate precision, the pipeline will pivot directly to YOLO Pose. Since our main goal is to accurately locate coordinates, keypoint detection is theoretically the superior choice for sub-pixel centering.

2. Image Size Strategy

- Step 6 - Standard YOLO Detection Model Choice:** Trained at **imgsz=1024**. Downsampling a 4096 x 2730 image to standard 640px size squashes a 40px target marker down to an un-learnable 6-pixel blur. Setting the input resolution to 1024px preserves edge definitions, allowing the network to distinguish shapes while remaining within GPU memory limits.
- Step 7 - YOLO Pose Model Choice:** Trained at **imgsz=1280**. Since keypoint regression requires maximum pixel-level structural details to find the exact center cross-hair or vertex, we upsampled the image canvas further to 1280px. This extra resolution ensures that fine details do not blur out during convolutional downsampling.

3. Synthetic Bounding Box Size Selection

Step 8 - Sizing Definition: Enforced a synthetic box size of **60 x 70 pixels** around the absolute center coordinates. Because the original JSON contains only individual point coordinates, a bounding box width and height must be synthetically generated to initialize anchor boxes for the object detection head. A tight 60 x 70 px envelope balances two needs: it provides the model with enough regional context to learn the surrounding ground features while remaining tight enough to prevent the box loss math from overfitting to surrounding soil, rocks, or quarry debris.

4. Train / Validation Data Split

Step 9 - Splitting Definition: Enforced a strict **Stratified 80/20 Train/Validation Split** over the 871 synchronized image-label pairs. Simple random splitting is highly risky when a dataset has a severe class imbalance (such as our L-Shape class dominating at ~49%). A random split could leave the validation set without enough minority class examples (Cross). Stratification ensures that the 80/20 data ratio is maintained identically across every individual shape class, protecting validation metrics from bias.

5. Post-Baseline Evaluation & Architectural Pivot to YOLO Pose

Step 10 - Baseline Limitations: The baseline YOLOv8s model found the markers reasonably well (mAP50 = 0.546), but its overall Precision sat at a low **0.464**, and its strict localization score (mAP50-95) was a weak **0.270**. The Cross class suffered from heavy false positives (Precision: 0.215) due to ground cracks mimicking boundaries. Most importantly, deriving coordinates mathematically from the center of a bounding box is highly volatile—any slight shift in the predicted outer box edge due to dust or shadows directly pulls the center coordinate away from the true pixel location.

Step 11 - The Final Pivot Outcomes (YOLOv8n-Pose): Because the baseline metrics were insufficient for survey-grade precision, we pivoted to direct keypoint regression. By explicitly targeting the center point independently of box edge alignment, the model achieved a massive performance leap: Overall Pose mAP50 hit **0.657** (a net +6.0% gain), and strict coordinate precision (Pose mAP50-95) reached an elite **0.640**, mathematically proving sub-pixel center coordinate accuracy.

Step 12 - Conclusion: Cross precision skyrocketed to **0.866** because the keypoint head explicitly searches for the central geometric intersection pixel, completely ignoring background soil cracks or tyre tracks. This direct point regression eliminated baseline errors while maintaining a compact file footprint of 6.5 MB and an edge-ready processing latency of 7.7ms per frame, making it fully ready for production drone deployment.